

#1: CS1 Review

Goals for today's class: CS1 Review

- Functions
 - ▶ parameters/arguments
 - ▶ return statements
- Conditionals
 - ▶ boolean expressions
 - ▶ if-else statements
 - ▶ nested/chained conditionals
- Loops
 - ▶ for loops with `range(·)`
 - ▶ while loops
- Lists
 - ▶ creating a list
 - ▶ accessing a list element
- Loop patterns
 - ▶ min/max, counter, quantifier ("for all"/"for some")

Outline

- 1 **Functions**
- 2 Conditionals
- 3 Loops
- 4 Lists
- 5 Loop Patterns

Functions with a return value

- Most of the time you'll write functions that return a value.

Example (function with return value)

```
def distance(x1,y1,x2,y2):  
    u, v = (x2-x1)**2, (y2-y1)**2  
    return math.sqrt(u+v)  
  
print(distance(0,0,3,4))    # prints 5
```

Functions with no return value

- Sometimes a function has **no return value** – it just prints from inside instead.
- A function that never hits `return` gives back `None` automatically.

Example (function without return value)

```
def distance(x1,y1,x2,y2):
    u, v = (x2-x1)**2, (y2-y1)**2
    print(math.sqrt(u+v))    # prints 5 here

distance(0,0,3,4)           # no return value
print(distance(0,0,3,4))    # prints None
```

if-else statements

Syntax

```
if boolean_expression:    # (either True or False)
    statements
else:                      # else block may be omitted
    statements
```

```
def f(x,y):
    if x > 0:
        print("positive")
    else:
        print("zero/negative")

    if x > 0 and y > 0:
        print("both positive")
```

Chained if-elif-else

- `elif b2` runs when `b1` is `False` and `b2` is `True`; only the **first matching branch**, top to bottom, runs.

```
def f(x):  
    if x > 0:  
        print("positive")  
    elif x < 0:  
        print("negative")  
    else:  
        print("zero")
```

Nested vs. flat (pruning)

- A nested if can often be flattened by **pruning** – the two forms below are equivalent.

nested

```
def absoluteValue(x):  
    if x < 0:  
        return -x  
    else:  
        return x
```

flat (pruned)

```
def absoluteValue(x):  
    if x < 0:  
        return -x  
    return x
```


Boolean functions

- Useful for hiding a complicated test inside an if.

```
def onePositive(x,y,z):  
    if x > 0 and y <= 0 and z <= 0:  
        return True  
    if x <= 0 and y > 0 and z <= 0:  
        return True  
    if x <= 0 and y <= 0 and z > 0:  
        return True  
    return False  
  
if onePositive(x,y,z):  
    print("Exactly one number is positive.")
```

Tip: return the boolean expression

- A boolean expression is already a value, so `if...:return True/else:return False` collapses to `return boolean_expr`.

before

```
def lessThan(x,y):  
    if x < y:  
        return True  
    else:  
        return False
```

after

```
def lessThan(x,y):  
    return x < y
```

Tip: `drop == True / == False`

- `b == True` is just `b`; `b == False` is **not** `b` – this also protects you from typing `=` when you meant `==`.

```
def singleDigit(x):  
    return 0 <= x < 10
```

```
# instead of singleDigit(a) == True and singleDigit(b) == False
```

```
if singleDigit(a) and not singleDigit(b):  
    ...
```

for loops with range

- `range(m,n)` is short for `range(m,n,1)`; `range(n)` is short for `range(0,n,1)`.

```
for k in range(n):
    print(k)                # 0 1 ... n-1
```

```
for k in range(1, n+1):
    print(k)                # 1 2 ... n
```

```
for k in range(10, 21, 2):
    print(k)                # 10 12 14 16 18 20
```

```
for k in range(n, 0, -1):
    print(k)                # n n-1 ... 2 1
```

Accumulating a sum or product

```
def sum(n):  
    x = 0                # identity element for addition is 0  
    for i in range(1,n+1):  
        x += i  
    return x  
  
def factorial(n):  
    x = 1                # identity element for multiplication is 1  
    for i in range(1,n+1):  
        x *= i  
    return x
```

Nested for loops

- Nested loops build 2-D output, like a multiplication table.

```
for i in range(1, 10):  
    for j in range(1, 10):  
        print(i*j, end=" ")  
    print()
```

while loops

- Repeats as long as a boolean expression holds.

```
def countDigits(n):  
    counter = 0  
    while n > 0:  
        counter += 1  
        n = n // 10  
    return counter  
  
print(countDigits(713))    # 3
```

Creating / accessing a list

- Indices run from 0 to `len(·)-1` (**not** starting at 1).

```
number = [2, 5, 8, 11, 14]
```

```
number = [None] * 5    # [None, None, None, None, None]
```

```
number[0] = 2
```

```
for i in range(len(number)):
    number[i] = 3*i + 2
```


Lists as function input/output

```
def func(b):  
    a = [None] * len(b)  
    for i in range(len(a)):  
        a[i] = b[i] + 1  
    return a  
  
num = [2, 4, 3, 1, 7, 2, 5, 6]  
c = func(num)  
print(c)      # [3, 5, 4, 2, 8, 3, 6, 7]
```

1: Minimum/maximum pattern

- You may need the **index** of the extreme value, not just the value itself.

```
def findMin(numbers):  
    min = numbers[0]  
    for i in range(len(numbers)):  
        if numbers[i] < min:  
            min = numbers[i]  
    return min
```

```
def findMinIndex(numbers):  
    minIndex = 0  
    for i in range(len(numbers)):  
        if numbers[i] < numbers[minIndex]:  
            minIndex = i  
    return minIndex
```

2: Counter pattern

- Count how many elements satisfy some condition.

```
def countPrime(numbers):  
    counter = 0  
    for i in range(len(numbers)):  
        if isPrime(numbers[i]):    # boolean function  
            counter += 1  
    return counter
```

3: Quantifier pattern ("for some" / "for all")

- By De Morgan's laws, "for all" can be built as the negation of "for some".

```
def somePositive(numbers):  
    for i in range(len(numbers)):  
        if numbers[i] > 0:  
            return True  
    return False
```

```
def allPositive(numbers):  
    for i in range(len(numbers)):  
        if not (numbers[i] > 0):    # numbers[i] <= 0  
            return False  
    return True
```

Quantifier pattern: primality test

- A "for all" pattern you'll see a lot this semester: "p is prime" $\equiv$ "p%i != 0 for all i".

```
def isPrime(p):  
    if p < 2:  
        return False  
    for i in range(2, p//2+1):  
        if not (p%i != 0):    # p%i == 0  
            return False  
    return True
```